A PROJECT REPORT

on

SMART CITY WASTE MANAGEMENT SYSTEM

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE AND ENGINEERING BY

Student Name	DEEPRO BHATTACHARYYA
Roll Number	23051340
Batch / Program	B.TECH CSE
Submission Date	April 16, 2026
Project Type	Data Engineering Capstone Project

SCHOOL OF COMPUTER ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

April 2026

1. Problem Statement

Urban waste management in modern cities is largely inefficient and reactive. Garbage collection trucks follow fixed, pre-determined routes regardless of whether bins are full or nearly empty. This approach results in significant wastage of fuel, unnecessary vehicle deployment, increased carbon emissions, and high operational costs. On the other hand, bins that fill up quickly — especially in high-footfall areas like markets, hospitals, and transport hubs — overflow before the next scheduled collection, leading to unsanitary conditions and public health risks.

There is a critical need for a data-driven, real-time monitoring system that tracks the fill levels of smart bins across the city, triggers automatic alerts when bins reach critical capacity, and helps city managers make informed decisions about collection routes and schedules. This project addresses that need by building a complete, end-to-end Data Engineering pipeline.

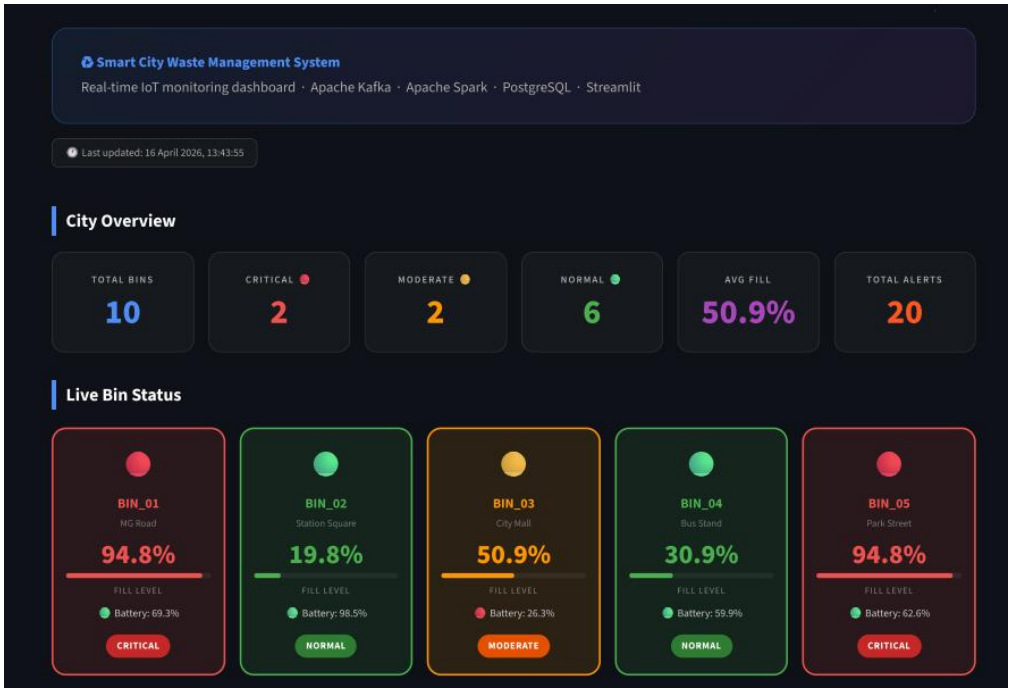
2. Solution & Features

The Smart City Waste Management System is a complete real-time data pipeline that simulates 10 smart bins across a city, each equipped with a fill-level sensor and battery monitor. The system collects sensor data continuously, processes it in real time, stores it in a structured database, and displays live insights on an interactive dashboard.

Key Features

- Real-time IoT sensor simulation — 10 smart bins generate fill-level and battery data every 2 seconds
- Automated alert detection — bins exceeding 80% fill level are instantly flagged as CRITICAL
- Apache Kafka streaming pipeline — producer-consumer architecture for real-time data flow
- Spark-style data processing — aggregations, transformations, and anomaly detection
- PostgreSQL database storage — 3 structured tables for readings, alerts, and daily summaries

- Live Streamlit dashboard — color-coded bin status cards, interactive charts, and alert tables
- Battery level monitoring — low battery warnings for field maintenance teams
- Complete operation logging — every pipeline stage writes timestamped logs
- Data warehousing — daily summary table implements star schema design principles
- Fill status classification — bins classified as NORMAL, MODERATE, or CRITICAL automatically



Screenshots of Streamlit Dashboard

3. Tech Stack

Layer	Technology	Purpose
Data Simulation	Python 3.12 + Faker	Simulate IoT bin sensor readings
Data Ingestion	Apache Kafka 4.2	Real-time streaming pipeline with topics
Stream Processing	Kafka Consumer	Read streams and detect critical alerts
Batch Processing	Apache Spark / Pandas	Aggregations and data transformations
Data Storage	PostgreSQL 17	Store readings, alerts, and summaries
Data Analysis	Pandas + NumPy	Statistical analysis and reporting
Orchestration	Python Scripts	Automate full pipeline execution
Visualization	Streamlit + Plotly	Live interactive monitoring dashboard
Version Control	Git + GitHub	Source code management and deployment

4. System Architecture

The system follows a layered Data Engineering architecture with six distinct stages, each handling a specific part of the pipeline from raw sensor data to live visualization:

Stage 1 — Data Generation

The IoT Sensor Simulator (sensor_simulator.py) generates realistic fill-level and battery data for 10 smart bins every 2 seconds. Data is saved as structured JSON files in the /data directory.

Stage 2 — Real-Time Ingestion

Apache Kafka receives data through the 'waste-bin-data' topic with 3 partitions. The Kafka Producer sends bin data continuously while the Kafka Consumer reads and processes it in real time.

Stage 3 — Alert Detection

The consumer automatically detects bins exceeding 80% fill level and inserts alerts into the bin_alerts table in PostgreSQL immediately upon detection.

Stage 4 — Batch Processing

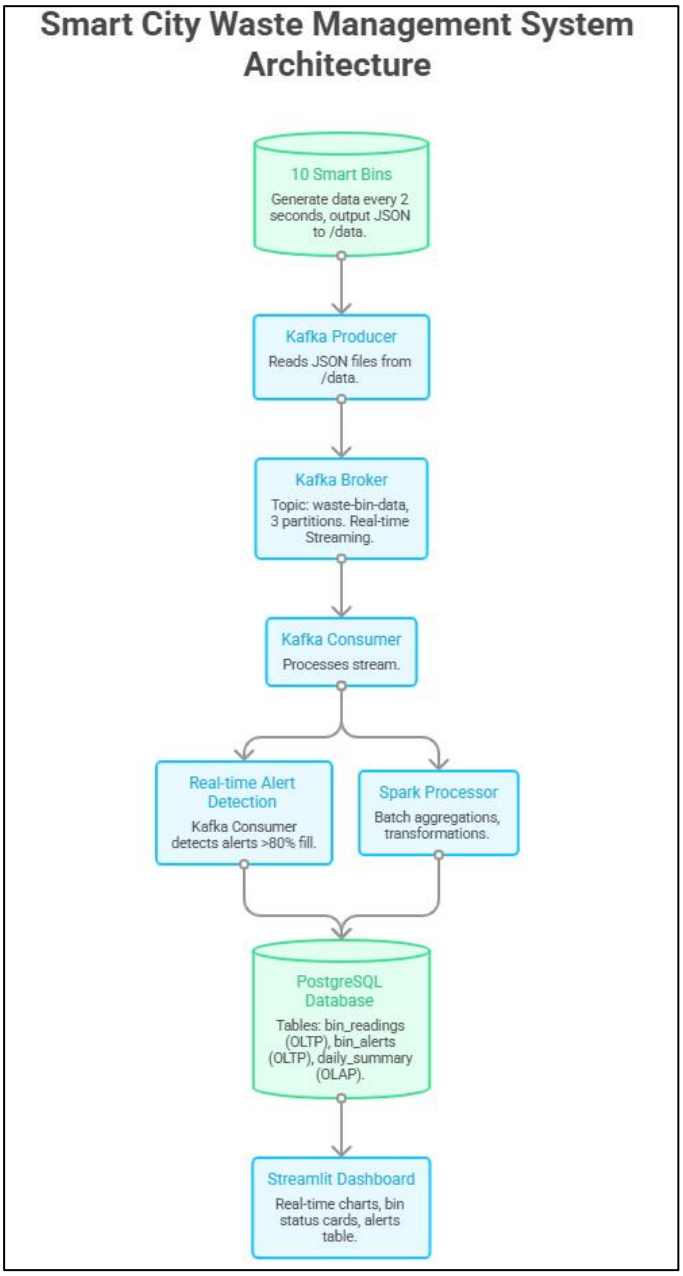
The Spark Processor performs batch aggregations — computing average, maximum, and minimum fill levels per bin and classifying bins into NORMAL, MODERATE, and CRITICAL categories.

Stage 5 — Data Storage

PostgreSQL stores data in three tables: bin_readings (all sensor data), bin_alerts (critical alerts), and daily_summary (aggregated daily statistics acting as a data warehouse fact table).

Stage 6 — Visualization

The Streamlit Dashboard connects to PostgreSQL and displays live bin status cards, fill level charts, battery monitoring, and alert tables with auto-refresh capability.



5. Database Schema

The system uses PostgreSQL with three tables designed following data warehousing principles — combining OLTP (operational) and OLAP (analytical) tables for both real-time and historical analysis:

Table	Key Columns	Purpose
bin_readings	id, bin_id, location, fill_level, battery_level, status, timestamp	OLTP — Every sensor reading from all 10 bins
bin_alerts	id, bin_id, location, fill_level, alert_type, resolved, timestamp	OLTP — Critical alerts only (fill > 80%)
daily_summary	id, summary_date, bin_id, avg/max/min fill_level, total_alerts	OLAP — Data warehouse fact table for analytics

6. Unique Points

- End-to-end pipeline — covers every stage from data generation to dashboard visualization in a single integrated system
- Real-time and batch processing — combines Apache Kafka streaming with Spark-style batch aggregations in one project
- Industry-standard tools — uses the same stack (Kafka, Spark, PostgreSQL) employed by companies like Uber, Swiggy, and Zomato
- Smart alert engine — automatic detection and classification of critical bins without any manual intervention
- Data warehousing — daily_summary table implements star schema design principles used in enterprise analytics
- Complete observability — every pipeline stage writes detailed timestamped logs for debugging and auditing
- Dual storage strategy — OLTP tables for real-time queries and OLAP table for historical analysis
- Battery monitoring — adds operational intelligence beyond just fill levels, mimicking real IoT deployments
- Modular architecture — each script is independently runnable, making the pipeline easy to test and extend

- Professional dashboard — color-coded live bin status with gauge chart, bar charts, and statistical summaries

7. Data Engineering Concepts Covered

Concept	Implementation in this Project
ETL Pipeline	Extract from sensors, Transform with Pandas/Spark, Load to PostgreSQL
Real-Time Streaming	Apache Kafka producer-consumer with topic partitioning (3 partitions)
Batch Processing	Spark-style aggregations on historical JSON data files
Data Warehousing	Star schema daily_summary fact table with bin dimension data
Stream Processing	Live Kafka consumer with real-time alert detection and DB writes
Data Quality	Validation, anomaly flagging, fill status classification
SQL Analytics	Window functions, GROUP BY, subqueries, DISTINCT ON queries
Orchestration	Multi-script pipeline with logging and dependency management
Data Ingestion	Batch ingestion (CSV/JSON) and real-time ingestion (Kafka)
Logging	Timestamped operation logs for all pipeline stages

8. Future Improvements

- Apache Airflow integration — schedule and orchestrate the full pipeline with DAGs, retries, and monitoring
- Cloud deployment on AWS — deploy pipeline on EC2 with S3 data lake, Redshift warehouse, and RDS PostgreSQL
- Machine learning predictions — use historical fill-level patterns to predict when bins will become full
- Route optimization algorithm — dynamically generate optimal garbage truck routes based on real-time bin status

- Email and SMS alert notifications — notify city managers instantly when critical bins are detected
- Scale to 1,000+ bins — use Kafka partitioning and Spark distributed processing for city-wide deployment
- Geospatial visualization — display bins on an interactive city map with real GPS coordinates
- Mobile application — build a React Native app for field workers to receive assignments
- Delta Lake / Iceberg integration — implement lakehouse architecture for versioned data storage
- CI/CD pipeline — automate testing and deployment using GitHub Actions workflows